A Layered Framework for Accessing AI Reasoning Data: APIs, Instrumentation, and Anchor Literature

Nathan Conklin
nathan.conklin@vt.edu
Department of Computer Science,
Virginia Tech

Abstract

Interactive visualization of AI reasoning depends on access to reasoning data, and that access varies by provider, model, and system layer. This paper organizes available access points into a six-layer framework: mechanistic interpretability, inference execution, reasoning representation, agent execution, evidence and provenance, and decision and verification. The layers are complementary views rather than a strict containment hierarchy. For each layer, we define what it captures, identify the APIs or software mechanisms that expose its data, and anchor it to seminal or survey literature. We then survey a provider landscape in which closed vendors generally withhold raw chain of thought but may expose summaries, opaque reasoning state, token probabilities, tool traces, or citations. We examine self-hosted open-weight models as an alternative that permits local instrumentation, subject to architecture, license, and infrastructure constraints. A targeted review of XAI, LLM interpretability, and agentic-transparency literature did not identify a taxonomy spanning all six views; this absence claim is bounded by the search described in Section 4. We ground the framework in a worked study of Kimi K3, including its schemas, trace-preservation semantics, reference capture code, and the practical cost of layer 1 and layer 2 access at 2.8-trillion-parameter scale.

1. Introduction

Reasoning models now generate intermediate work products before answering: chains of thought, tool invocations, retrieved evidence, and self-checks. Chain-of-thought prompting established that eliciting intermediate reasoning steps improves performance on multi-step tasks [1], and subsequent model generations internalized the technique. Interfaces that support human sensemaking over this reasoning need the underlying data, yet the data is scattered across system layers and gated behind inconsistent access policies.

The access problem has two dimensions. First, commercial providers restrict what they return. Major APIs do not expose raw internal chain of thought, but their observable surfaces are not limited to summaries: depending on the provider and model, a developer may receive no readable reasoning, a synthesized summary, opaque or encrypted reasoning state, output-token probabilities, tool-execution events, citations, or a more detailed generated trace. Second, even where a trace is available, it is one artifact among several. A complete account of why a system produced an answer spans internal computation, the emitted rationale, agent actions, retrieved evidence, and validation results.

This paper contributes a six-layer access framework that organizes these data sources from model internals outward to application-level checks, pairs each layer with its access mechanism, and grounds each layer in the literature. The numbering describes analytic distance from the model, not a monotonic access order: an application may expose layers 4 through 6 while withholding layers 1 through 3. The framework supports a design argument for interactive visual chain-of-thought research: rather than treating any single trace as ground truth, an interface should record a unified provenance graph across layers and let the user move between complementary perspectives.

Section 2 defines the six layers with their access mechanisms and anchor papers. Section 3 surveys the provider API landscape and presents a worked instrumentation study of one frontier open-weight model. Section 4 maps the framework

onto three research communities that partially cover it. Section 5 discusses faithfulness limits and open-weight deployment, and Section 6 concludes.

2. The Six-Layer Access Framework

The framework orders data sources from model internals outward to application-level checks. Layers 1 through 3 describe model-layer data, layers 4 and 6 describe agent-layer behavior, and layer 5 describes the knowledge layer. These are complementary views rather than nested levels of permission or capability; access to an outer layer does not imply access to an inner one.

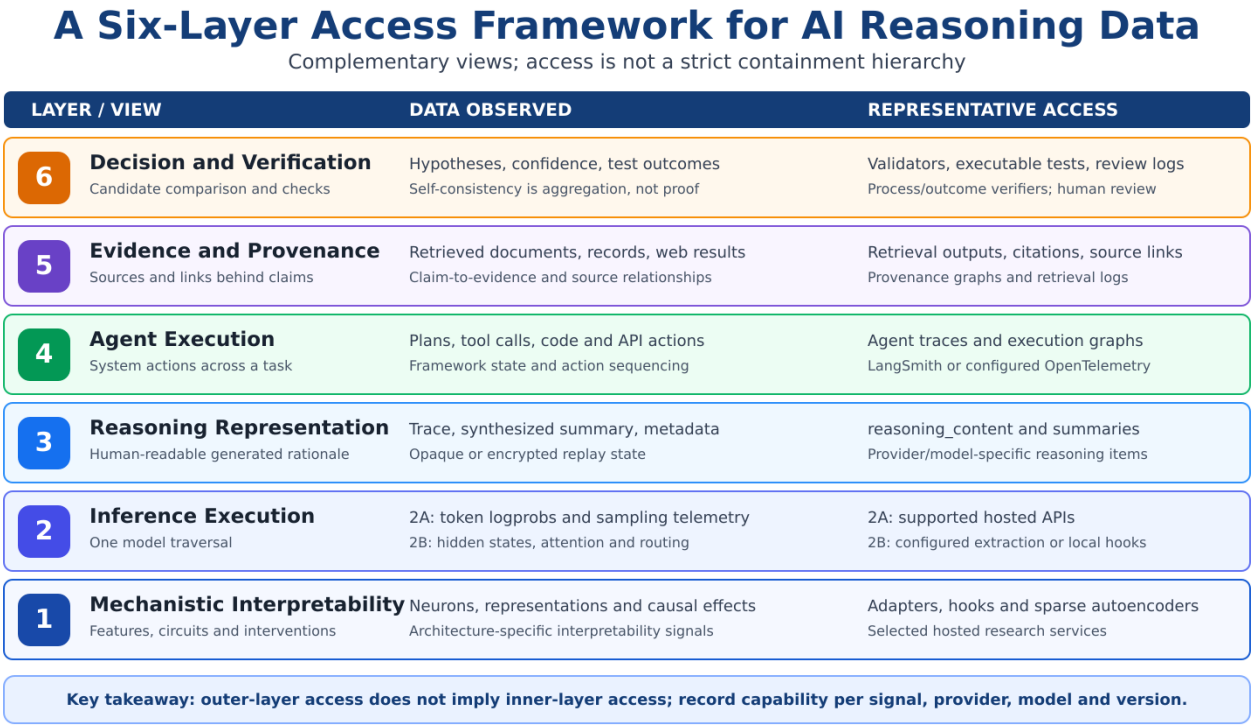

Figure 1: The six-layer framework for accessing AI reasoning data, with representative technologies and access mechanisms at each layer.

2.1 Layer 1: Mechanistic Interpretability

Definition. How the neural network computes internally: neurons, circuits, and feature representations. This is a research method rather than a standard production-model API. Hosted interpretability platforms exist, but major frontier-model providers do not provide general mechanistic access to the internals of their production frontier models.

Access mechanism. Interpretability tooling applied to locally or remotely instrumented models. TransformerLens 3 uses architecture adapters and TransformerBridge to support many Hugging Face architecture families, while sparse-autoencoder pipelines extract interpretable features from activations [10], [34]. Coverage of a newly released architecture such as Kimi K3 still depends on an available adapter or custom integration.

Anchor literature. Anthropic's Scaling Monosemanticity demonstrated feature extraction from a production-scale model and is foundational for the feature-level view [9]. Sharkey et al. survey the field's methods and open problems [8].

Hosted service status. Neuronpedia has operated an interpretability API since March 2024, providing feature dashboards, steering controls, activation and circuit inspection, probes, and sparse-autoencoder resources over selected models; the platform is open source [29]. Goodfire's Ember provided programmatic feature inspection and steering for selected models, including Llama 3.3 70B [24]. Public availability and supported-model lists can change, so provider documentation should be checked at the time of a study. These services do not establish general layer 1 access to frontier models such as Kimi K3. The largest example discussed here, 70B versus 2.8T parameters, is approximately a 40-fold difference rather than two to three orders of magnitude.

Tooling constraints. Instrumentation still depends on model architecture even though current tools are broader than their early GPT-only implementations. TransformerLens 3's TransformerBridge claims support across thousands of Hugging Face models and roughly fifty architecture families through architecture-specific adapters [34]. NNsight builds an intervention graph over PyTorch modules and its companion NDIF service enables remote inspection of selected hosted models [25]. Neither tool guarantees immediate support for a novel attention or mixture-of-experts implementation; Kimi K3 may require a custom adapter and model-specific hook discovery. A minimal NNsight activation capture and causal patching experiment takes the following form:

```python
from nnsight import LanguageModel

model = LanguageModel("Qwen/Qwen3-8B", device_map="auto", dispatch=True)

# Capture: residual stream, attention output, and logits at one forward pass.
with model.trace("The capital of France is"):
    resid = model.model.layers[20].output[0].save()   # (batch, seq, d_model)
    attn = model.model.layers[20].self_attn.output[0].save()
    logits = model.lm_head.output.save()

# Causal intervention: patch a corrupted activation into a clean run.
with model.trace() as tracer:
    with tracer.invoke("The Eiffel Tower is in Rome"):
        corrupted = model.model.layers[15].output[0].save()
    with tracer.invoke("The Eiffel Tower is in Paris"):
        model.model.layers[15].output[0][:] = corrupted
        patched_logits = model.lm_head.output.save()
```

Feasibility at frontier scale. Applying this tooling to a 2.8-trillion-parameter model is not a matter of hardware capacity alone. Kimi K3 has 104 billion active parameters per token and 93 layers comprising 69 Kimi Delta Attention layers and 24 Gated Multi-head Latent Attention layers; it has 896 experts and selects 16 experts per token, with one dense layer and MoE routing in the remaining expert layers [26]. At four bits per weight, the parameters occupy approximately 1.4 TB before runtime overhead. That exceeds typical H100-class workstations and servers, but not every current single node: an eight-GPU NVIDIA DGX B300 provides more than 2 TB of GPU memory [35]. Activation capture, caches, framework overhead, and experimental replication may still require multi-node infrastructure. The custom architecture also requires model-specific hook discovery. A defensible research program can conduct mature layer 1 methods on smaller supported models, then treat frontier-scale layer 1 work as a separately resourced effort rather than assuming that open weights make it routine.

2.2 Layer 2: Inference Execution

Definition. Observable and internal data produced during a specific inference run. This layer has two subtypes: layer 2A, black-box sampling telemetry such as output-token log probabilities; and layer 2B, internal tensors and control signals such as hidden states, attention patterns, and mixture-of-experts routing decisions.

Access mechanism. Layer 2A may be available through hosted APIs: OpenAI Chat Completions and the Gemini API expose output-token log probabilities on supported models [36], [37]. Layer 2B normally requires self-hosted instrumentation, a specialized hosted research service, or hooks attached to the inference stack [12]. Production serving engines such as vLLM define the stack that a local deployment can instrument [13]. This subdivision avoids treating limited sampling telemetry as equivalent to direct model-state access.

Anchor literature. The Transformer architecture paper supplies the mechanics being observed [11]; vLLM's PagedAttention paper anchors the serving-systems side [13].

What serving engines actually expose. Running an open-weight model on vLLM [13] or SGLang [27] does not automatically provide all layer 2B data. vLLM added native hidden-state extraction in version 0.18.0 [30]. The server must be launched with the documented speculative-decoding and KV-transfer configuration; a request then uses kv_transfer_params and writes selected hidden states to safetensors files. Request configuration alone is insufficient. The feature targets speculative-decoder data generation, writes states to disk rather than the response, and does not expose attention matrices or expert-routing decisions. Current vLLM documentation identifies chunked prefill as incompatible with the extraction path and instructs users to disable it [30]. An older third-party implementation reports automatic-prefix-caching defects in the earlier speculators generator [32], but that report should not be attributed to the native vLLM 0.18 extraction system. A general request-time activation interception interface remains an open proposal [31]. Open weights therefore establish the precondition for full layer 2B access without making every signal available through an unmodified serving engine.

Expert routing as a tractable layer 2 signal. Sparse mixture-of-experts models offer a compact layer 2B signal. Each MoE router computes a discrete top-k expert selection per token, following the sparse-routing pattern established by Switch Transformers and successors [28]. Kimi K3 selects sixteen of 896 experts in each MoE layer; because the architecture includes one dense layer, routing should not be described as occurring in all 93 layers [26]. Integer routing records are far smaller than dense activations and can be aligned causally with emitted tokens, provided the analysis accounts for the next-token offset between an input position's state and the token it predicts. Whether routing trajectories systematically distinguish reasoning phases remains an empirical hypothesis, not an established visualization result.

2.3 Layer 3: Reasoning Representation

Definition. The model's own human-readable rationale, if it emits one: chain of thought or a summary of it. This is a generated artifact, distinct from the computation in layers 1 and 2.

Access mechanism. Model-specific and provider-specific. Hosted APIs may return no readable reasoning, synthesized summaries, opaque or encrypted reasoning state, or richer generated traces [2], [3], [38]. Open-weight reasoning models can emit reasoning tokens directly [4], [6], [7]. Availability depends on model training, endpoint policy, and model version.

Anchor literature. Chain-of-thought prompting is the classic [1]. Zhao et al.'s ACM survey of LLM explainability provides breadth across explanation techniques for this layer [14], and Turpin et al. establish the faithfulness caveat [5].

Internal structure of an emitted trace. Where a provider does return a trace, the returned artifact is typically flat. Kimi K3's `reasoning_content` is undifferentiated text: no step boundaries, no markers separating deliberation from tool-call planning, no confidence annotations [7]. Any decomposition into the structured reasoning representation an interactive interface needs is client-side work, performed by segmentation over prose the model never intended to be parsed. The one structural boundary the API supplies arrives only in streaming mode, where the transition from `reasoning_content` deltas to `content` deltas separates deliberation from answer generation. Recording the timestamp of that transition yields a deliberation duration distinct from generation duration, which is a measurable per-turn quantity that non-streaming access discards.

Preservation semantics and their design consequences. Providers differ in what they return and what state a client must replay. In stored OpenAI Responses workflows, opaque reasoning items can be retained and referenced by identifier; in stateless or zero-data-retention workflows, encrypted_content can be returned to the client and replayed so the service can decrypt it transiently. Neither mode exposes plaintext internal chain of thought [2]. Anthropic behavior is model-dependent: some current models return summarized thinking, some can omit readable thinking while preserving signed or encrypted state, and newer Opus generations differ from older models in whether earlier thinking blocks are retained [3]. Kimi K3 requires the caller to replay the complete assistant message on later turns, including reasoning_content and any tool_calls [7],

[26]. A client that stores only final content can silently lose information required for best multi-turn behavior. Interfaces should therefore record provider, model version, disclosure mode, and replay semantics with every trace.

Trace-level intervention. Kimi K3 supports two distinct mechanisms that should not be conflated. First, a client can edit a previously stored reasoning_content field, replay that assistant turn as conversation history, and ask the model to reason again; the model card demonstrates that caller-supplied reasoning history can influence a later response [26]. This is a history-level intervention and should be evaluated experimentally rather than described as access to the model's original internal computation. Second, partial mode continues generation from a supplied assistant content prefix [7]. Partial mode does not, by itself, make an edited reasoning_content field a continuation prefix. The following example demonstrates the first mechanism and intentionally does not claim partial-mode behavior:

```
# History-level intervention: replay an edited trace in a later turn.
edited_history = [
    {"role": "user", "content": original_question},
    {
        "role": "assistant",
        "reasoning_content": analyst_revised_reasoning,
        "content": original_answer,
    },
    {
        "role": "user",
        "content": "Re-derive the conclusion using the revised rationale.",
    },
]

completion = client.chat.completions.create(
    model="kimi-k3",
    messages=edited_history,
    reasoning_effort="max",
)

# This tests history replay. Kimi partial mode is a separate mechanism
# for continuing from an assistant answer-content prefix.
```

This distinguishes an editable history experiment from a cosmetic annotation. With K3, an edited rationale can be replayed as input and its downstream effect measured, but the resulting behavior does not prove that the text corresponds to internal computation. On providers that expose only summaries or opaque state, the interface may annotate the visible representation without being able to replay a modified plaintext rationale. Trace mutability is therefore endpoint- and model-specific, and studies should distinguish annotation, history replay, and true activation-level intervention.

Interaction with structured output and tool calling. Two further API behaviors bear on layer 3 capture. Constraining the response with a strict JSON schema shapes the final `content` only and leaves `reasoning_content` unconstrained [7], so an interface can obtain machine-readable answers without the schema distorting the deliberation it is trying to study. In tool-calling loops, the caller returns the complete assistant message and one tool result per call before requesting continuation [7], which accumulates a separate reasoning segment per tool step. Agentic execution thus supplies, as a side effect, the trace segmentation that single-turn generation withholds, and it aligns each segment with a layer 4 action.

2.4 Layer 4: Agent Execution

Definition. What the overall system does: plans, tool calls, code execution, and API invocations across a multi-step task.

Access mechanism. The orchestration layer. LangGraph supports tracing through its LangSmith integration, which must be enabled, while Semantic Kernel emits OpenTelemetry-compatible logs, metrics, and traces when telemetry and exporters are configured [16], [17], [40], [41]. These frameworks make layer 4 operationally accessible, but trace completeness still depends on instrumentation settings and on what external tools return.

Anchor literature. ReAct established the interleaved reasoning-and-acting pattern that agent traces record [15]. Raza et al.'s survey of transparency in agentic AI covers the trace, oversight, and governance literature for this layer [18].

2.5 Layer 5: Evidence and Provenance

Definition. Where information came from: retrieved documents, database records, and web results, together with the links between that evidence and the answer.

Access mechanism. Retrieval frameworks, vector databases, search APIs, and Model Context Protocol servers, which standardize tool and data-source connections for LLM applications [20]. Provenance capture happens at the application boundary where these components return results.

Anchor literature. Retrieval-augmented generation grounds the evidence-linking pattern [19]. XAI taxonomies treat provenance as a component of explanation [23], and the agentic transparency literature extends it to governance [18].

2.6 Layer 6: Decision and Verification

Definition. The meta-level checks: hypothesis comparisons, confidence estimates, and validation tests over candidate answers.

Access mechanism. Application-level. The developer implements self-checks or validators and logs their results; no standard API exists.

Anchor literature. Self-consistency samples multiple reasoning paths and aggregates their answers, making it a canonical decision-layer technique rather than an independent correctness guarantee [21]. Verification research distinguishes outcome verifiers that score complete solutions from process verifiers that judge intermediate steps. Cobbe et al. trained outcome verifiers to rank candidate mathematical solutions [42]; Uesato et al. compared process- and outcome-based feedback [43]; and Lightman et al. established step-level process supervision at larger scale [22]. Application verification can also include executable tests, schema checks, evidence-entailment checks, constraint validation, and human review. These mechanisms should be logged separately because agreement among correlated model-based judges is not equivalent to independent verification.

3. The Provider API Landscape

Commercial reasoning APIs converge on partial rather than uniform disclosure. OpenAI does not return raw plaintext chain of thought, but its Responses API can return reasoning summaries and either stored opaque reasoning items or client-held encrypted_content for replay, depending on the data-retention mode [2]. Anthropic's readable-thinking and preservation behavior varies by model generation [3]. Google Gemini exposes synthesized thought summaries on supported thinking models and can expose output log probabilities through its generation API, but not raw internal chain of thought [37], [38]. Provider rationales for limiting disclosure differ and include user experience, safety and monitoring concerns, protection of model capabilities, and defenses against unauthorized distillation; these should be attributed to the specific provider rather than generalized as a single industry justification.

Open-weight models provide a different access path. DeepSeek-R1 emits reasoning-style traces as part of its output [4], although such traces are not guaranteed to faithfully report the computation that produced the answer [5]. GLM-5.2, released by Zhipu AI under an MIT license, is a mixture-of-experts model with approximately 750 billion total parameters and about 40 billion active per token [6]. Its API supports High and Max thinking modes and returns a dedicated reasoning_content field [39]. Kimi K3 runs with thinking enabled and returns a distinct reasoning_content field alongside answer content through an OpenAI-compatible API [7]. These are generated representations, not direct observations of hidden computation.

Self-hosting an open-weight model permits local capture of emitted reasoning tokens and model instrumentation that typical closed hosted APIs do not provide. The achievable depth still depends on the license, model implementation, serving configuration, hardware, and availability of architecture-specific adapters. An emitted trace remains one generated signal among several and should not be treated as the model's true internal reasoning [5].

3.1 Case Study: The Kimi K3 Reasoning Interface

Kimi K3 merits detailed treatment because it occupies the permissive end of the disclosure spectrum while remaining a public, documented, commercially served API. Studying it shows what a provider gives away when it declines to withhold the trace.

Moonshot serves K3 at `https://api.moonshot.ai/v1` under the model string `kimi-k3`, and publishes both OpenAI-compatible and Anthropic-compatible surfaces [26]. A researcher can therefore point an existing OpenAI SDK client at the endpoint without rewriting the request layer. Access requires a successful account top-up of at least one dollar, and cumulative top-up determines the concurrency, RPM, TPM, and TPD limits an account receives [7].

The minimal request differs from a standard chat completion only in the top-level reasoning field:

```
curl https://api.moonshot.ai/v1/chat/completions \
  --header "Authorization: Bearer $MOONSHOT_API_KEY" \
  --header "Content-Type: application/json" \
  --data '{
    "model": "kimi-k3",
    "reasoning_effort": "max",
    "messages": [
      {"role": "user", "content": "Prove that the square root of 2 is irrational."}
    ]
  }'
```

K3 keeps thinking mode enabled on every request; the documentation states that callers cannot disable it and can use reasoning_effort to alter deliberation depth [7]. This makes a reasoning_content channel consistently available through the documented endpoint. It does not establish that every emitted trace is meaningful, necessary, or faithful to the computation that produced the answer.

The three effort tiers reached the API in stages, and a study that reports an effort setting should record both its date and its service. Moonshot's launch announcement stated that K3 would run at `max` effort by default with `low` and `high` to follow in subsequent updates; the additional tiers appeared in the documentation within days [7], [26]. Defaults also differ by endpoint: the OpenPlatform Chat Completions API described here defaults to `max`, while the managed Kimi Code service accepts the same three values and defaults to `high`. Moonshot's technical report documents how the three effort levels were produced during post-training, including the per-problem token budgets used to separate them [33].

The non-streaming response carries the trace in a `reasoning_content` field beside the answer:

```
{
  "id": "chatcmpl-...",
  "object": "chat.completion",
  "model": "kimi-k3",
  "choices": [{
    "index": 0,
    "message": {
      "role": "assistant",
      "reasoning_content": "Assume sqrt(2) = p/q in lowest terms. Then p^2 = 2q^2 ...",
      "content": "Suppose the square root of 2 were rational ...",
      "tool_calls": null
    },
    "finish_reason": "stop"
```

```
  }],
  "usage": {
    "prompt_tokens": 24,
    "completion_tokens": 1840,
    "total_tokens": 1864,
    "cached_tokens": 0
  }
}
```

Streaming splits the two channels into separate deltas [7]. Each server-sent event is a `chat.completion.chunk` whose `choices[0].delta` carries a `reasoning_content` fragment or a `content` fragment; in observed traffic the reasoning deltas arrive first and the channel switches once, though the documentation does not guarantee that a single chunk never carries both:

```
{"choices":[{"index":0,"delta":{"reasoning_content":"Assume"},"finish_reason":null}]}
{"choices":[{"index":0,"delta":{"reasoning_content":" p/q"},"finish_reason":null}]}
{"choices":[{"index":0,"delta":{"content":"Suppose"},"finish_reason":null}]}
```

The following request-level constraints govern any experiment built on the endpoint:

Field	Behavior	Consequence for study design
`reasoning_effort`	`low`, `high`, `max`; defaults to `max` on OpenPlatform and `high` on managed Kimi Code	The only exposed control over deliberation depth. Replaces the K2.x `thinking` parameter.
`temperature`, `top_p`, `n`, `presence_penalty`, `frequency_penalty`	Fixed server-side; requests must omit them	No sampling sweeps and no multi-sample generation in a single call.
`max_completion_tokens`	Defaults to 131072, settable to 1048576	Reasoning and answer tokens draw from one budget; long traces can starve the answer.
Assistant message replay	Complete message required on the next turn	Dropping `reasoning_content` degrades multi-turn quality with no error raised.
Vision input	Base64 or `ms://<file-id>` only; no public URLs	Image stimuli must be embedded or uploaded through the Files API.
Prefix caching	Automatic once a prior prompt exceeds 256 tokens	Repeated requests sharing an eligible prompt prefix remain candidates for automatic caching; measure cache hits from response telemetry.

3.2 Fixed Sampling and Its Cross-Layer Consequences

K3 pins temperature, top_p, n, and both penalty parameters to fixed values and rejects overrides [7]. Two research consequences follow, and both cut across the framework rather than sitting inside layer 3.

First, the fixed temperature removes the standard method for characterizing reasoning variance. A researcher cannot sweep temperature to observe how trace structure destabilizes, so variance studies must resample identical requests and treat the residual nondeterminism of the serving stack as the only source of variation. That is a weaker instrument, and it is a hosted-endpoint limitation rather than a model limitation; a self-hosted deployment restores full sampling control.

Second, n=1 prevents multi-sample generation within one API call, so a layer 6 self-consistency study must issue independent requests and assemble the candidate set client-side [21]. This changes request overhead and orchestration, but it does not inherently defeat prefix caching: requests that repeat an eligible prompt prefix can still use Moonshot's automatic

cache. The effective cost must therefore be measured from billed and cached-token telemetry rather than inferred from n alone.

4. Position in the Literature

In a targeted review, we did not identify a published taxonomy that exactly matches these six views. The conclusion is bounded rather than exhaustive: the review examined the XAI surveys, LLM explainability and mechanistic-interpretability surveys, agentic-transparency literature, and provider documentation cited here, using combinations of reasoning access, chain-of-thought access, interpretability taxonomy, agent trace, provenance, and verification through August 2026. XAI taxonomies organize explanation types broadly; Schwalbe and Finzel provide a systematic survey of surveys [23]. Zhao et al. survey LLM explainability [14], Sharkey et al. survey mechanistic interpretability [8], and Raza et al. discuss the gap between static-model XAI and agent-lifecycle transparency in an EngrXiv preprint [18]. A broader systematic review could still identify adjacent or overlapping taxonomies.

The proposed mapping is an author synthesis: mechanistic interpretability aligns with interpretability research, reasoning representation with LLM explainability, agent execution and verification with agentic transparency, and evidence and provenance with both XAI and agentic governance. The candidate contribution is not a proven absence of prior work, but a practical integration framework that treats all six layers as views over one AI response and makes their access mechanisms explicit.

5. Discussion

Emitted reasoning is one signal, not ground truth. A layer 3 trace may diverge from the computation that produced an answer [5]. Cross-layer comparison should therefore be operationalized claim by claim. For each answer claim, the system records linked trace segments, retrieved evidence, tool calls, available layer 2 telemetry, and verifier results in a shared event graph. Pre-registered comparisons then measure evidence-to-claim entailment, trace-to-answer consistency, trace-to-action consistency, verifier agreement, and stability across repeated samples. Human-coded evaluations should use at least two raters and report inter-rater agreement; model-based judges should be calibrated against held-out human labels. Hidden-state or routing associations remain exploratory until causal interventions or controlled ablations support them. This procedure does not make any signal ground truth, but it makes agreement and conflict observable and reproducible.

Open-weight deployment is an enabling strategy for layers 1 through 3, but not the only source of useful access. Hosted APIs can expose layer 2A sampling telemetry, layer 3 summaries or opaque state, and application layers 4 through 6. Self-hosting GLM-5.2 or Kimi K3 adds the possibility of direct reasoning-token capture and layer 2B or layer 1 instrumentation, subject to model architecture, license, serving support, and infrastructure. The framework should therefore record access capability per signal rather than assigning each provider a single maximum layer.

Design consequence for interactive visualization. The architecture should split the model layer, the agent layer, and the knowledge layer explicitly and record a unified provenance graph across them. An interface built over that graph can let a user traverse all perspectives on one response: from a claim in the answer, to the evidence behind it, to the tool call that fetched the evidence, to the reasoning step that requested it, and, where instrumentation allows, to the model internals active at that step. A durable version of this approach captures explicit reasoning graphs, evidence, and confidence as first-class application data rather than relying on private chains of thought that providers may withdraw at any time.

Provider parameter policy is a research constraint, not a detail. Fixed sampling parameters on a hosted endpoint eliminate temperature sweeps and require independent requests for self-consistency [7], [21]. Independent requests can still share an eligible cached prefix, so affordability should be measured using request, token, latency, and cache-hit telemetry. Self-hosting can restore parameter control, but only after the deployment reproduces the model's documented chat template and reasoning parser.

Trace mutability separates annotation, history replay, and intervention. Kimi K3 permits a client to replay modified reasoning_content as prior conversation state, while partial mode separately continues from an assistant answer-content prefix [7], [26]. OpenAI and Anthropic expose different combinations of summaries and opaque or encrypted state, and those combinations vary by data-retention mode or model version [2], [3]. A visual reasoning system should state which of the three interaction levels it supports and should not describe history replay as activation-level intervention.

Layer 1 access does not follow from open weights. Releasing weights removes a provider-access barrier, not every legal or engineering barrier; use remains subject to the model's license. Kimi K3's approximately 1.4 TB of four-bit weights can fit in a current B300-class eight-GPU node, but interpretability also needs memory for runtime state, activation capture, and experimental replication [26], [35]. TransformerLens now supports far more than GPT-style models through adapters, yet a novel Kimi architecture may still require custom integration [34]. Hosted services cover selected models rather than providing general frontier-scale access. Expert-routing capture is a promising compact layer 2B signal, but its feasibility and interpretive value on K3 must be demonstrated rather than assumed.

6. Conclusion

Access to AI reasoning data is layered, but the layers are complementary views rather than a strict permission hierarchy. The six-layer framework pairs mechanistic, inference, representational, agent, evidence, and verification data with their concrete access mechanisms and anchor literature. Three qualified findings stand out: closed providers withhold raw plaintext chain of thought while exposing different combinations of summaries, opaque state, sampling telemetry, tools, and citations; self-hosted open-weight models can enable deeper instrumentation when licenses, adapters, serving configuration, and infrastructure permit it; and the targeted literature review did not identify an exact six-view taxonomy. A unified cross-layer provenance graph can address the practical integration problem while supporting empirical tests of where reasoning traces, actions, evidence, and verification agree or conflict.

Appendix A. Reference Implementation: Trace Capture Client

The client below captures Kimi K3 reasoning traces, preserves complete assistant turns, records the transition between deliberation and answer generation, and aggregates streamed tool-call deltas. It targets Moonshot's OpenAI-compatible endpoint. A self-hosted vLLM or SGLang deployment may use the same client structure, but normally requires a deployment-specific base_url, model identifier or alias, API key policy, chat template, and reasoning parser; changing base_url alone is not guaranteed to be sufficient.

```
"""Capture Kimi K3 traces, phase timing, and complete assistant turns."""

import os
import time
from dataclasses import dataclass, field
from typing import Any, Iterator, Optional

from openai import OpenAI


@dataclass
class TraceRecord:
    """One assistant turn split into deliberation and answer phases."""

    turn_index: int
    reasoning_effort: str
    reasoning_content: str
    content: str
    tool_calls: list[dict[str, Any]] = field(default_factory=list)
    finish_reason: Optional[str] = None
    t_first_reasoning_token: Optional[float] = None
    t_first_content_token: Optional[float] = None
    t_complete: Optional[float] = None

    @property
    def deliberation_seconds(self) -> Optional[float]:
        if (
            self.t_first_reasoning_token is None
            or self.t_first_content_token is None
        ):
            return None
        return self.t_first_content_token - self.t_first_reasoning_token

    def to_assistant_message(self) -> dict[str, Any]:
        message: dict[str, Any] = {
            "role": "assistant",
            "reasoning_content": self.reasoning_content,
            "content": self.content,
        }
        if self.tool_calls:
            message["tool_calls"] = self.tool_calls
        return message


class K3TraceClient:
    """Retain the full reasoning and tool-call record for each turn."""

    def __init__(
```

```
        self,
        base_url: str = "https://api.moonshot.ai/v1",
        model: str = "kimi-k3",
        api_key: Optional[str] = None,
    ) -> None:
        resolved_key = api_key or os.getenv("MOONSHOT_API_KEY") or "not-needed"
        self.client = OpenAI(api_key=resolved_key, base_url=base_url)
        self.model = model
        self.messages: list[dict[str, Any]] = []
        self.traces: list[TraceRecord] = []

    def ask(
        self,
        prompt: str,
        reasoning_effort: str = "max",
        max_completion_tokens: int = 131072,
    ) -> TraceRecord:
        """Send one non-streaming turn and preserve the returned message."""
        self.messages.append({"role": "user", "content": prompt})
        start = time.perf_counter()
        completion = self.client.chat.completions.create(
            model=self.model,
            messages=self.messages,
            reasoning_effort=reasoning_effort,

            max_completion_tokens=max_completion_tokens,
        )
        choice = completion.choices[0]
        message = choice.message
        reasoning = (
            getattr(message, "reasoning_content", None)
            or getattr(message, "reasoning", None)
            or ""
        )
        record = TraceRecord(
            turn_index=len(self.traces),
            reasoning_effort=reasoning_effort,
            reasoning_content=reasoning,
            content=message.content or "",
            tool_calls=[
                tc.model_dump(exclude_none=True)
                for tc in (message.tool_calls or [])
            ],
            finish_reason=choice.finish_reason,
            t_complete=time.perf_counter() - start,
        )
        self.messages.append(message.model_dump(exclude_none=True))
        self.traces.append(record)
        return record
```

```
    def ask_streaming(
        self,
        prompt: str,
        reasoning_effort: str = "max",
        max_completion_tokens: int = 131072,
    ) -> Iterator[tuple[str, str]]:
        """Yield reasoning/content text and aggregate streamed tool calls."""
        self.messages.append({"role": "user", "content": prompt})
        start = time.perf_counter()
        record = TraceRecord(
            turn_index=len(self.traces),
            reasoning_effort=reasoning_effort,
            reasoning_content="",
            content="",
        )
        stream = self.client.chat.completions.create(
            model=self.model,
            messages=self.messages,
            reasoning_effort=reasoning_effort,
            max_completion_tokens=max_completion_tokens,
            stream=True,
        )

        reasoning_parts: list[str] = []
        content_parts: list[str] = []
        tool_parts: dict[int, dict[str, Any]] = {}

        for chunk in stream:
            if not chunk.choices:
                continue
            choice = chunk.choices[0]
            delta = choice.delta
            now = time.perf_counter() - start

            reasoning_delta = (
                getattr(delta, "reasoning_content", None)
                or getattr(delta, "reasoning", None)
            )
            if reasoning_delta:
                if record.t_first_reasoning_token is None:
                    record.t_first_reasoning_token = now
                reasoning_parts.append(reasoning_delta)
                yield ("reasoning", reasoning_delta)

            if delta.content:
                if record.t_first_content_token is None:
                    record.t_first_content_token = now
```

```
                content_parts.append(delta.content)
                yield ("content", delta.content)

            for tool_delta in (delta.tool_calls or []):
                entry = tool_parts.setdefault(
                    tool_delta.index,
                    {
                        "id": "",
                        "type": "function",
                        "function": {"name": "", "arguments": ""},
                    },
                )
                if tool_delta.id:
                    entry["id"] = tool_delta.id
                if tool_delta.type:
                    entry["type"] = tool_delta.type
                if tool_delta.function:
                    if tool_delta.function.name:
                        entry["function"]["name"] += tool_delta.function.name
                    if tool_delta.function.arguments:
                        entry["function"]["arguments"] += (
                            tool_delta.function.arguments
                        )

            if choice.finish_reason:
                record.finish_reason = choice.finish_reason

        record.reasoning_content = "".join(reasoning_parts)
        record.content = "".join(content_parts)
        record.tool_calls = [tool_parts[i] for i in sorted(tool_parts)]
        record.t_complete = time.perf_counter() - start
        self.messages.append(record.to_assistant_message())
        self.traces.append(record)
```

Two implementation points carry the correctness of the client. The non-streaming path appends message.model_dump(exclude_none=True), preserving non-standard fields such as reasoning_content and complete tool calls. The streaming path must explicitly aggregate fragmented tool-call identifiers, names, and argument strings before rebuilding the assistant message; ignoring those deltas makes later tool-result replay invalid. Because third-party gateways may expose reasoning_content as an untyped extra or rename it to reasoning, the code reads both forms. A production client should additionally validate the final message against the selected server's chat template and persist raw response events for auditability.

References

1. J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2201.11903. https://arxiv.org/abs/2201.11903.
2. OpenAI. Reasoning Models. Official OpenAI documentation. Accessed August 2026. https://developers.openai.com/api/docs/guides/reasoning.
3. Anthropic. Extended Thinking. Claude Platform Documentation. Accessed August 2026. https://platform.claude.com/docs/en/build-with-claude/extended-thinking.
4. DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948, 2025. https://arxiv.org/abs/2501.12948.
5. M. Turpin, J. Michael, E. Perez, and S. R. Bowman. Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.04388. https://arxiv.org/abs/2305.04388.

6. Zhipu AI (Z.ai). GLM-5.2. Open-weight model release, MIT license, June 13, 2026. https://huggingface.co/zai-org/GLM-5.2.
7. Moonshot AI. Kimi K3. Model release and API documentation, July 2026. https://platform.kimi.ai/docs/guide/kimi-k3-quickstart.
8. L. Sharkey, B. Chughtai, J. Batson, J. Lindsey, J. Wu, et al. Open Problems in Mechanistic Interpretability. *Transactions on Machine Learning Research*, 2025. arXiv:2501.16496. https://arxiv.org/abs/2501.16496.
9. A. Templeton, T. Conerly, J. Marcus, J. Lindsey, T. Bricken, et al. Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet. *Transformer Circuits Thread*, Anthropic, 2024. https://transformer-circuits.pub/2024/scaling-monosemanticity/.
10. N. Nanda and J. Bloom. TransformerLens. GitHub repository, 2022. https://github.com/TransformerLensOrg/TransformerLens.
11. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. arXiv:1706.03762. https://arxiv.org/abs/1706.03762.
12. A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. arXiv:1912.01703. https://arxiv.org/abs/1912.01703.
13. W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180. https://arxiv.org/abs/2309.06180.
14. H. Zhao, H. Chen, F. Yang, N. Liu, H. Deng, H. Cai, S. Wang, D. Yin, and M. Du. Explainability for Large Language Models: A Survey. *ACM Transactions on Intelligent Systems and Technology*, 15(2), Article 20, 2024. doi:10.1145/3639372. https://doi.org/10.1145/3639372.
15. S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629. https://arxiv.org/abs/2210.03629.
16. LangChain Inc. LangGraph. Accessed August 2026. https://www.langchain.com/langgraph.
17. Microsoft. Semantic Kernel. Accessed August 2026. https://learn.microsoft.com/en-us/semantic-kernel/.
18. S. Raza, A. Y. Radwan, S. Chaduvula, M. Alinoori, and C. Emmanouilidis. Transparency in Agentic AI: A Survey of Interpretability, Explainability, and Governance. *EngrXiv* preprint 6451, 2026. https://engrxiv.org/preprint/view/6451.
19. P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 33:9459-9474, 2020. arXiv:2005.11401. https://arxiv.org/abs/2005.11401.
20. Anthropic. Model Context Protocol. 2024. https://modelcontextprotocol.io.
21. X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2203.11171. https://arxiv.org/abs/2203.11171.
22. H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's Verify Step by Step. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2305.20050. https://arxiv.org/abs/2305.20050.
23. G. Schwalbe and B. Finzel. A Comprehensive Taxonomy for Explainable Artificial Intelligence: A Systematic Survey of Surveys on Methods and Concepts. *Data Mining and Knowledge Discovery*, 38(5):3043-3101, 2024. doi:10.1007/s10618-022-00867-8. https://doi.org/10.1007/s10618-022-00867-8.
24. Goodfire. Announcing Goodfire Ember. Goodfire Blog, December 23, 2024. https://www.goodfire.ai/blog/announcing-goodfire-ember.
25. J. Fiotto-Kaufman, A. R. Loftus, E. Todd, J. Brinkmann, K. Pal, et al. NNsight and NDIF: Democratizing Access to Open-Weight Foundation Model Internals. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2407.14561. https://arxiv.org/abs/2407.14561.

26. Moonshot AI. Kimi-K3 Model Card. Hugging Face, July 2026. https://huggingface.co/moonshotai/Kimi-K3.
27. SGLang Team. SGLang: A Fast Serving Framework for Large Language Models. GitHub repository, accessed August 2026. https://github.com/sgl-project/sglang.
28. W. Fedus, B. Zoph, and N. Shazeer. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research*, 23(120):1-39, 2022. arXiv:2101.03961. https://arxiv.org/abs/2101.03961.
29. J. Lin and J. Bloom. Neuronpedia: An Open Platform for Interpretability Research. Accessed August 2026. https://www.neuronpedia.org/.
30. vLLM Project. Hidden State Extraction. vLLM Documentation, feature introduced in v0.18.0 (PR #33736), March 2026. https://docs.vllm.ai/en/stable/features/speculative_decoding/extract_hidden_states/.
31. vLLM Project. RFC: Observation Plugin for Intercepting and Routing on Activations. GitHub issue #36998, March 2026. https://github.com/vllm-project/vllm/issues/36998.
32. Agency Enterprise. vllm-hidden-states: Extract Hidden States from Intermediate Transformer Layers Using vLLM. GitHub repository, accessed August 2026. https://github.com/agencyenterprise/vllm-hidden-states.
33. Moonshot AI. Kimi K3 Technical Report. July 2026. https://github.com/MoonshotAI/Kimi-K3/blob/main/k3_tech_report.pdf.
34. TransformerLens. TransformerBridge and supported-model documentation. Accessed August 2026. https://transformerlensorg.github.io/TransformerLens/content/getting_started.html.
35. NVIDIA. NVIDIA DGX B300 System Specifications. Accessed August 2026. https://docs.nvidia.com/dgx/dgxb300-user-guide/introduction-to-dgxb300.html.
36. OpenAI. Using logprobs. Official OpenAI Cookbook, December 20, 2023; accessed August 2026. https://developers.openai.com/cookbook/examples/using_logprobs.
37. Google. Gemini API: GenerateContent response and log-probability fields. Accessed August 2026. https://ai.google.dev/api/generate-content.
38. Google. Gemini API I/O updates: thought summaries. Google Developers Blog. Accessed August 2026. https://developers.googleblog.com/gemini-api-io-updates/.
39. Z.ai. Thinking Mode and reasoning_content. Z.ai Developer Documentation. Accessed August 2026. https://docs.z.ai/guides/capabilities/thinking.
40. LangChain. Trace a LangGraph application. LangSmith Documentation. Accessed August 2026. https://docs.langchain.com/langsmith/trace-with-langgraph.
41. Microsoft. Observability in Semantic Kernel. Accessed August 2026. https://learn.microsoft.com/en-us/semantic-kernel/concepts/enterprise-readiness/observability/.
42. K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, et al. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168, 2021. https://arxiv.org/abs/2110.14168.
43. J. Uesato, N. Kushman, R. Kumar, F. Song, N. Siegel, et al. Solving Math Word Problems with Process- and Outcome-Based Feedback. arXiv:2211.14275, 2022. https://arxiv.org/abs/2211.14275.